What is Memory Management in Python?

Memory management means **how Python uses and controls computer memory**
👉 Memory management का मतलब है **Python कैसे memory को use और control करता है**

Python **automatically gives and removes memory**
👉 Python खुद से memory देता और हटाता है

We don't need to manage memory manually like C/C++
👉 हमें manually memory manage नहीं करनी पड़ती

Python uses **Garbage Collection**
👉 Python unused objects को हटाने के लिए Garbage Collector use करता है

How Python Manages Memory

1️⃣ **Memory Allocation**

- When we create a variable, Python gives memory automatically
  👉 जब हम variable बनाते हैं, Python खुद memory allocate करता है

2️⃣ **Reference Counting**

- Python counts how many variables are pointing to an object
  👉 Python देखता है कितने variables उस object को point कर रहे हैं
- If reference becomes 0, memory is removed
  👉 अगर count 0 हो जाए, memory free हो जाती है

3️⃣ **Garbage Collector**

- Removes unused or circular objectsHow do you optimize **slow APIs**?
  👉 Unused या circular objects को हटाता है
- Runs automatically in background
  👉 Background में automatically चलता है

What is garbage collection?

⍰ **Garbage Collection (GC)** is a process that **removes unused objects from memory**
👉 Garbage Collection एक process है जो **unused objects को memory से हटा देता है**

⍰ It helps free memory automatically
👉 यह memory को automatically free करता है

⍰ Python does this in the background
👉 Python इसे background में खुद करता है

What is reference counting?

- **Reference counting** is a memory management technique in Python.
- Python counts how many variables are pointing to an object.
- If the count becomes **0**, the object is deleted from memory.

👉 Reference counting Python की memory management technique है।
👉 Python गिनता है कितने variables किसी object को point कर रहे हैं।
👉 अगर count 0 हो जाए, तो object memory से हट जाता है।

What are magic methods?

- **Magic methods** are special methods in Python.
- They start and end with double underscores __.
- They define how objects behave.
- Also called **dunder methods** (double underscore).

👉 Magic methods Python के special methods होते हैं।
👉 ये __ से शुरू और खत्म होते हैं।
👉 ये object का behavior define करते हैं।

Common Examples

- __init__() → Constructor (object बनाते समय चलता है)
- __str__() → String representation
- __len__() → Length return करता है
- __add__() → + operator को control करता है
- __eq__() → Equality check (==)

What is asyncio?

asyncio is a Python library used for **asynchronous programming**.

⍰ It helps run **multiple tasks at the same time** without blocking the program.

⍰ It is mainly used for **I/O tasks** like API calls, database queries, file reading.

⍰ It uses async and await keywords.

⍰ It is efficient for handling **many users or requests**.

- **asyncio** Python की एक library है जो **asynchronous programming** के लिए use होती है।
- यह program को **block किए बिना** multiple tasks चलाने देती है।
- Mostly API, database, file operations में use होती है।
- इसमें async और await keywords use होते हैं।
- यह ज्यादा users को efficiently handle करने में मदद करती है।

What is Dependency Injection?

Dependency Injection means giving a class the things it needs from outside.
👉 Dependency Injection का मतलब है class को उसकी जरूरत की चीजें बाहर से देना।

- A class should not create its own dependencies.
  👉 Class को अपनी dependency खुद नहीं बनानी चाहिए।
- Instead, we pass the dependency from outside.
  👉 Instead, dependency को बाहर से pass किया जाता है।
- It makes the code easier to test and change.
  👉 इससे code को test और change करना आसान हो जाता है।
- It reduces tight connection between classes.
  👉 यह classes के बीच की tight connection को कम करता है।

What is Docker and how Python app runs in Docker?

Docker is a tool that helps run applications inside containers.
👉 Docker एक tool है जो applications को container के अंदर run करने में मदद करता है।

A container packages the app with all its dependencies.
👉 Container app को उसकी सभी dependencies के साथ package करता है।

It works the same on every system.
👉 यह हर system पर same तरीके से काम करता है।

It solves “it works on my machine” problem.
👉 यह “मेरे system पर तो चल रहा है” वाली problem को solve करता है।

It is lightweight and fast compared to virtual machines.
👉 यह virtual machine से हल्का और तेज होता है।

What is CI/CD?

CI/CD is a process that automates building, testing, and deploying code.
👉 CI/CD एक process है जो code को build, test और deploy automatically करता है।

CI means Continuous Integration.
👉 CI का मतलब Continuous Integration है।

CD means Continuous Deployment or Continuous Delivery.
👉 CD का मतलब Continuous Deployment या Continuous Delivery है।

It helps developers release code faster and safely.
👉 यह developers को code जल्दी और safely release करने में मदद करता है।

It reduces manual work and human errors.
👉 यह manual काम और human errors को कम करता है।

What is deadlock in Python?

Deadlock happens when two or more threads are waiting for each other forever.
👉 Deadlock तब होता है जब दो या ज्यादा threads एक-दूसरे का इंतजार करते रहते हैं।

It usually happens when using locks in multithreading.
👉 यह अक्सर multithreading में locks इस्तेमाल करते समय होता है।

Each thread holds a lock and waits for another lock.
👉 हर thread एक lock पकड़ता है और दूसरे lock का इंतजार करता है।

Because of this, the program stops and does not continue.
👉 इसकी वजह से program रुक जाता है और आगे नहीं बढ़ता।

It is a common problem in concurrent programming.
👉 यह concurrent programming में एक common problem है।

How do you improve performance of slow Python code?

First, identify the bottleneck using profiling tools like cProfile.
👉 पहले profiling tools (जैसे cProfile) से पता करें कि code कहाँ slow है।

Optimize algorithms and data structures.
👉 बेहतर algorithm और data structure का use करें।

Avoid unnecessary loops and repeated calculations.
👉 Extra loops और बार-बार होने वाली calculation को हटाएँ।

Use built-in functions and libraries because they are faster.
👉 Built-in functions और libraries का use करें क्योंकि वे तेज होती हैं।

Use list comprehensions instead of normal loops when possible.
👉 Possible हो तो normal loop की जगह list comprehension use करें।

Use multiprocessing or multithreading for parallel tasks.
👉 Parallel tasks के लिए multiprocessing या multithreading use करें।

Use caching for repeated results.
👉 बार-बार आने वाले results के लिए caching का उपयोग करें।

Optimize database queries and reduce I/O operations.
👉 Database queries optimize करें और I/O operations कम करें।

Use async programming for I/O heavy tasks.
👉 I/O heavy tasks के लिए async programming use करें।

Consider using C extensions or tools like NumPy for heavy computation.
👉 Heavy computation के लिए C extensions या NumPy जैसे tools use करें।

How do you structure a large Python project?

Keep the project modular by dividing it into multiple packages and modules.
👉 Project को छोटे packages और modules में divide करें।

Separate business logic, database, and API layers.
👉 Business logic, database और API layers को अलग रखें।

Use a clear folder structure (like app, tests, config, utils).
👉 Clear folder structure रखें (जैसे app, tests, config, utils)।

Use virtual environments and requirements.txt.
👉 Virtual environment और requirements.txt का उपयोग करें।

Write unit tests and keep them in a separate tests folder.
👉 Unit tests लिखें और उन्हें अलग tests folder में रखें।

Use environment variables for configuration.
👉 Configuration के लिए environment variables use करें।

Follow coding standards (PEP 8).
👉 PEP 8 coding standard follow करें।

Use logging instead of print statements.
👉 Print की जगह logging का उपयोग करें।

Document the project with README and comments.
👉 README और comments के साथ project को document करें।

How do you secure a Python API?

Use authentication like JWT or OAuth.
👉 JWT या OAuth जैसी authentication use करें।

Use HTTPS to encrypt data in transit.
👉 Data encryption के लिए हमेशा HTTPS use करें।

Validate and sanitize all user inputs.
👉 User input को validate और sanitize करें।

Use proper authorization (role-based access).
👉 Role-based authorization लागू करें।

Limit API requests using rate limiting.
👉 Rate limiting से request limit करें।

Store secrets in environment variables.
👉 Secret keys को environment variables में रखें।

Enable logging and monitoring.
👉 Logging और monitoring enable करें।

Handle errors properly without exposing sensitive data.
👉 Error handling ऐसा करें कि sensitive data expose न हो।

Keep dependencies updated.
👉 Libraries और dependencies को update रखें।

How do you handle large file processing?

Do not load the entire file into memory at once.
👉 पूरी file को एक साथ memory में load न करें।

Read the file in chunks.
👉 File को छोटे-छोटे chunks में पढ़ें।

Use generators to process data step by step.
👉 Data को step by step process करने के लिए generators का उपयोग करें।

Use streaming instead of full download when possible.
👉 Possible हो तो full download की जगह streaming करें।

Use multiprocessing for CPU-heavy tasks.
👉 CPU-heavy task के लिए multiprocessing का उपयोग करें।

Use async I/O for large network or disk operations.
👉 Large network या disk operations के लिए async I/O use करें।

Store temporary data efficiently (avoid large lists).
👉 Temporary data को efficient तरीके से store करें (बड़ी lists avoid करें)।

Use optimized libraries like pandas for large datasets.
👉 Large data के लिए optimized libraries (जैसे pandas) का उपयोग करें।

What challenges have you faced in your Python project?

Handling performance issues when the application became slow.
👉 जब application slow हो गया तो performance issue handle करना।

Managing database optimization and slow queries.
👉 Database optimization और slow queries को manage करना।

Fixing bugs related to concurrency (threading/async).
👉 Threading या async से जुड़े bugs को fix करना।

Handling large file or large data processing efficiently.
👉 Large file या data को efficiently process करना।

Managing environment and dependency conflicts.
👉 Environment और dependency conflicts को solve करना।

Implementing proper error handling in production.
👉 Production में proper error handling implement करना।

Securing APIs from unauthorized access.
👉 API को unauthorized access से secure करना।

Deploying the app and handling server-related issues.
👉 App deploy करते समय server issues handle करना।